

VITYARTHI BUILD YOUR OWN PROJECT

Name: **SARWAGYA KUMAR**

Registration Number: **26MIP10067**

Program: **Integrated M.Tech Computer Science and Engineering (Computational and Data Science)**

Course: **Problem Solving in Python(CSE-1021)**

Faculty: **Siddharth Singh Chouhan (SCAI)**

Academic Year: **2026-27**

VIT BHOPAL UNIVERSITY

INTRODUCTION

Cricket is a popular sport in which the performance of a player can be measured using different batting and bowling statistics. A player's performance may vary from match to match, making it useful to calculate overall records such as total runs, highest score, batting average, strike rate, fifties, hundreds and total wickets.

The Cricket Player Record and Statistics System is a basic Python-based project developed to calculate and display these important statistics. The program takes the player's name, team, role and number of matches as input. It then collects match-wise information such as runs scored, balls faced and wickets taken.

The entered data is processed using basic Python concepts such as variables, arithmetic operations, “for” loops and conditional statements. The system calculates the player's batting and bowling statistics and also analyzes the player's overall performance as Excellent, Good, or Needs Improvement.

The main purpose of this project is to demonstrate how basic programming concepts can be applied to a real-world cricket statistics problem. The final output is presented as a simple and organized player performance report.

Problem Statement

Cricket players participate in many matches and produce different batting and bowling performances in each match. Manually calculating their total runs, highest score, batting average, strike rate, fifties, hundreds and wickets can be time-consuming and may lead to calculation errors.

The Cricket Player Record and Statistics System is developed to solve this problem. The system takes the player's basic information and match-wise performance as input and automatically calculates important batting and bowling statistics.

The system also analyzes the player's average performance and displays an overall performance level as Excellent, Good, or Needs Improvement.

Functional Requirements

1. Player Information output .

- Player name
- Palyer's team
- Player's Role
- Number of matches palyed

2. Match Performance Input

- Runs scored
- Balls faced
- Wickets taken

3. Batting Statistics Calculation

- Total runs
- Highest score
- Batting average
- Strike rate
- Number of fifties
- Number of hundreds

4. Bowling Statistics Calculation

- Total wickets
- Best wickets taken in one match

5. Performance Analysis

- Runs per match
- Wickets per match

Based on these values, the system shall classify the player's overall performance as:

- Excellent
- Good
- Needs Improvement

Non-Functional Requirements

1. **Usability:** The program should be simple and easy to use. The user should be able to enter player and match information through clear input messages.
2. **Performance:** The program should calculate the statistics quickly after the user enters the match information.
3. **Reliability:** The program should correctly calculate the statistics based on the values entered by the user.
4. **Maintainability:** The program should be simple and easy to understand so that changes or improvements can be made easily.
5. **Resource Efficiency:** The program should use minimal computer resources because it performs only basic calculations and does not require external software or databases.
6. **Error Handling:** The program checks whether total balls are greater than zero before calculating the strike rate. If total balls are zero, the strike rate is set to zero.

System Architecture

The system follows a simple Input → Processing → Output architecture.

Input

The user provides:

- Player information
- Number of matches
- Runs scored in each match
- Balls faced in each match
- Wickets taken in each match

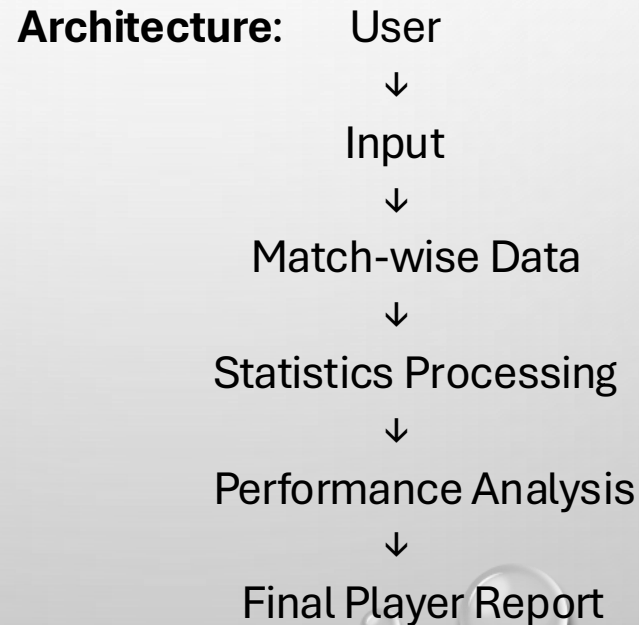
Processing

The program processes the entered information and calculates:

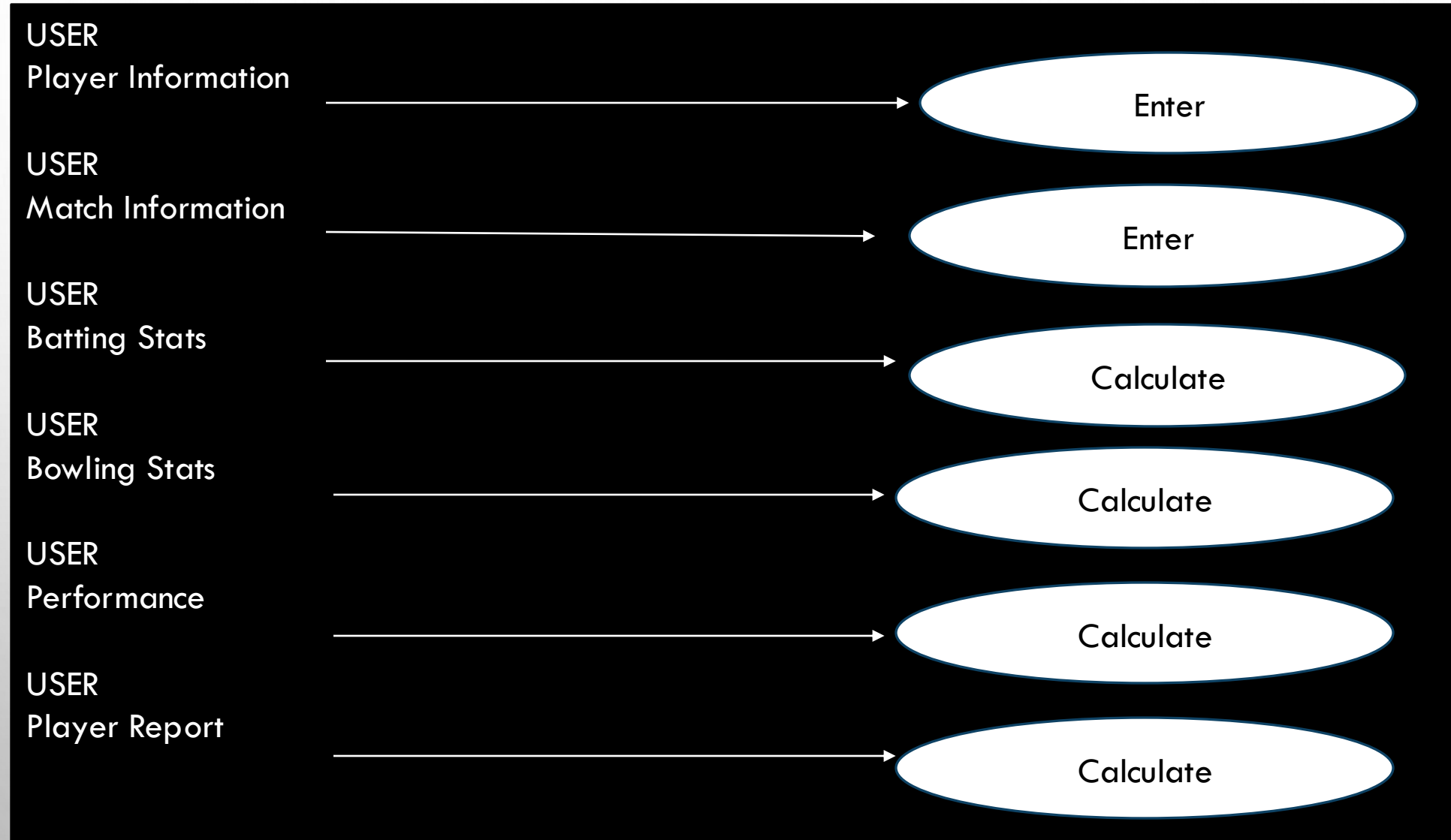
- Total runs
- Total balls
- Highest score
- Fifties
- Hundreds
- Batting average

- Best wickets
- Runs per match
- Wickets per match
- Overall performance

Output: The system displays a final player performance report containing the player's information, batting statistics, bowling statistics and overall performance.



USER DIAGRAM



WORKFLOW

START



Player Information



Number of Matches



Match-wise Input



Statistics Calculation



Performance Analysis

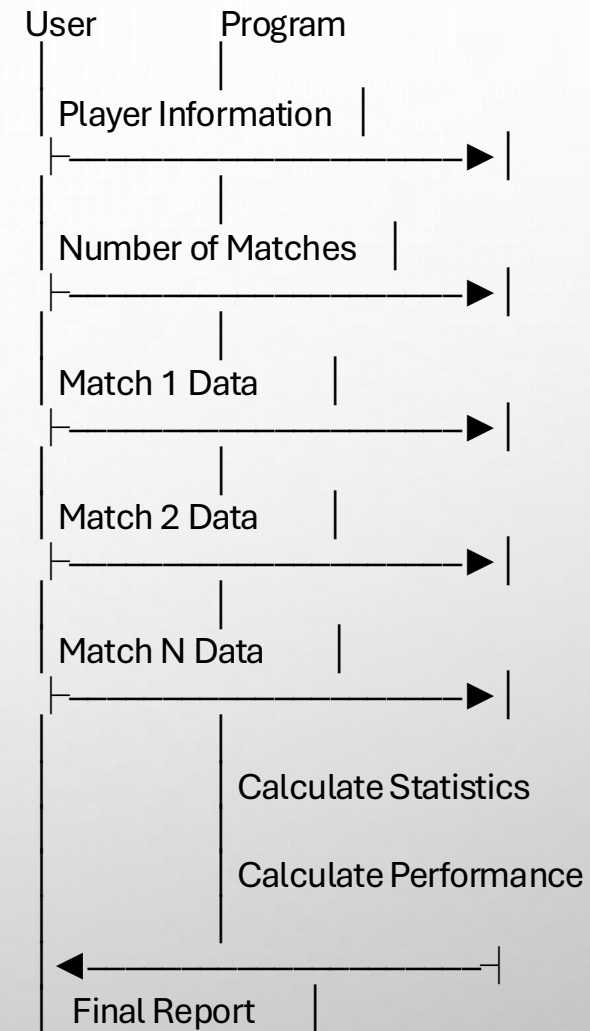


Player Report



END

SEQUENCE DIAGRAM



IMPLEMENTATION DETAILS

The Cricket Player Record and Statistics System is implemented using basic Python programming concepts. The program takes player information and match-wise performance as input, processes the data using arithmetic operations and conditional statements, and displays the final player statistics.

1. Programming Language

The project is implemented using Python. Python was selected because it provides simple syntax and is suitable for implementing basic calculations and data-processing operations.

2. Input

The program first collects the basic information of the cricket player:

- Player name
- Player's team
- Player's role
- Number of matches played

For every match, the program takes:

- Runs scored
- Number of balls faced
- Number of wickets taken

The `input()` statement is used to collect the information from the user, while `int()` is used to convert numerical inputs into integers.

3. Initialization of Variables

The program initializes the statistical variables with zero before processing the match data.

The main variables used are:

- `Total_runs`
- `total_balls`
- `highest_score`
- `fifties`
- `hundreds`
- `total_wickets`
- `best_wickets`

4. Match-wise Data Processing

A for loop is used to process the information for every match.

For each match, the program accepts the player's runs, balls faced and wickets taken.

The values are then added to the corresponding total variables.

For example:

`total_runs = total_runs + runs`. Similarly, total balls and total wickets are calculated.

5. Finding Highest Score and Best Wickets

The program uses if statements to find the player's highest score and the highest number of wickets taken in a single match.

For example:

`if runs > highest_score:`

`highest_score = runs`. The same method is used to find the best wickets in one match.

6. Counting Fifties and Hundreds

The program checks the runs scored in each match. If the player scores 50 or more runs, the number of fifties is increased.

If the player scores 100 or more runs, the number of hundreds is increased.

This is implemented using conditional statements:

`if runs >= 50:`

`fifties = fifties + 1`

`if runs >= 100:`

`hundreds = hundreds + 1`

7. Batting Statistics

After all matches have been processed, the program calculates the batting average using:

Batting Average = Total Runs / Matches Played

The strike rate is calculated using:

Strike Rate = (Total Runs / Total Balls) × 100

The program also checks whether the total number of balls is greater than zero before calculating the strike rate. If the total balls are zero, the strike rate is set to zero.

8. Bowling Statistics

The program calculates:

Total wickets

Best wickets in one match

The total wickets are obtained by adding the wickets taken in each match.

The if statement is used to identify the highest number of wickets taken in a single match.

9. Performance Analysis

The program calculates:

Runs Per Match = Total Runs / Matches Played

Wickets Per Match = Total Wickets / Matches Played

These values are used to determine the player's overall performance.

The program uses conditional statements:

- If runs per match are at least 40 and wickets per match are at least 2 → Excellent
- If runs per match are at least 30 or wickets per match are at least 1 → Good
- Otherwise → Needs Improvement

10. Output

After all calculations are completed, the program displays a structured player report.

The report contains:

Player Information

- Player name
- Team
- Role
- Matches played

Batting Statistics

- Total runs
- Highest score
- Batting average
- Strike rate
- Number of 50s
- Number of 100s

Bowling Statistics

- Total wickets
- Best wickets in one match

Performance

- Overall performance rating

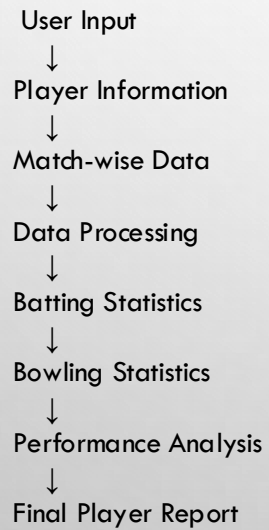
11. Programming Concepts Used

The project uses the following basic Python concepts:

- Variables
- `input()`
- `print()`
- Integer conversion using `int()`
- Arithmetic operations
- for loop
- if, elif and else statements
- Comparison operators
- Basic calculations

No external libraries, database or advanced Python features are used in the current implementation.

12. Overall Implementation Flow



Test Case

Test Case| Calculate cricket player's statistics

Player Name| Virat Kohli

Team| India

Role| Batsman

Number of Matches| 3

Match 1 Runs| 50

Match 1 Balls| 60

Match 1 Wickets| 0

Match 2 Runs| 100

Match 2 Balls| 110

Match 2 Wickets| 0

Match 3 Runs| 30

Match 3 Balls| 40

Match 3 Wickets| 0

Expected Total Runs| 180

Expected Highest Score| 100

Expected 50s| 2

Expected 100s| 1

Expected Total Wickets| 0

Expected Result| Statistics displayed correctly

Status| Pass

TEST CASE RESULT

CRICKET PLAYER RECORD AND STATISTICS

Enter Your Favourite Player name: Virat Kohli

Enter Palyer's team: India

Enter Player's Role: Batsman

Enter number of matches: 3

Match 1

Enter Player's Runs: 50

Enter no. of balls faced: 60

Enter no. of wickets taken: 0

Match 2

Enter Player's Runs: 100

Enter no. of balls faced: 110

Enter no. of wickets taken: 0

Match 3

Enter Player's Runs: 30

Enter no. of balls faced: 40

Enter no. of wickets taken: 0

PLAYER'S RECORD

Player Name: Virat Kohli

Team: India

Role: Batsman

Matches Played: 3

BATTING STATISTICS

Total Runs: 180

Highest Score: 100

Batting Average: 60.0

Strike Rate: 85.71428571428571

Number of 50s: 2

Number of 100s: 1

BOWLING STATISTICS

Total Wickets: 0

Best Wickets in One Match: 0

PERFORMANCE

Overall Performance: Good

END OF PROGRAM

[Program finished]